

Page 1: Node.js Fundamentals

Introduction

Node.js is a JavaScript runtime built on the V8 JavaScript engine. It allows JavaScript to run outside a web browser, making it useful for servers, APIs, command-line tools, automation, and backend applications. Node.js uses an event-driven architecture and is especially effective for applications that perform many input and output operations.

Runtime Model

A Node.js application normally runs in a single main JavaScript thread. Instead of creating a new thread for every request, Node.js uses an event loop and asynchronous APIs. Long-running I/O work such as network calls, file operations, and database access can be handled without blocking the main JavaScript execution path.

Why Node.js is popular

Node.js has a large npm ecosystem, fast startup, good support for HTTP services, and allows teams to use JavaScript or TypeScript across frontend and backend systems. It is commonly used for REST APIs, microservices, real-time applications, backend-for-frontend services, and event-driven systems.

Basic application

A minimal HTTP server can be created with the built-in http module. The server receives a request, executes application logic, and sends a response. In production applications, frameworks such as Express or NestJS are often used to organize routing, middleware, validation, and other concerns.

Page 2: Node.js Event Loop

Event-driven architecture

The event loop is a central part of Node.js. JavaScript code executes on the main thread while asynchronous operations can be delegated to the operating system or Node.js internal mechanisms. When an operation completes, its callback or continuation becomes eligible to run through the event loop.

Blocking versus non-blocking

A blocking operation prevents JavaScript from continuing until the operation finishes. A non-blocking operation starts work and allows other code to execute while waiting. Backend applications should avoid unnecessary synchronous operations in request handlers because blocking work can reduce throughput.

Promises and async/await

Modern Node.js applications commonly use Promises and async/await. An async function returns a Promise. The await keyword pauses that async function until the Promise settles, but it does not block the entire Node.js process. Proper error handling with try and catch is important when using await.

CPU-intensive work

Node.js is strongest for I/O-bound workloads. CPU-heavy tasks can block the event loop. For expensive computation, applications can use worker threads, separate services, queues, or other architecture choices so that request handling remains responsive.

Page 3: Modules and npm

CommonJS and ES Modules

Node.js supports CommonJS and ECMAScript modules. CommonJS commonly uses `require` and `module.exports`. ES Modules use `import` and `export`. New projects should choose a module system deliberately and keep the project configuration consistent.

npm

npm is the package manager commonly used with Node.js. A `package.json` file describes project metadata, scripts, dependencies, and development dependencies. `package-lock.json` records resolved dependency versions so installations can be reproduced more consistently.

Dependencies

Production dependencies are packages required by the running application. Development dependencies are typically tools used for testing, linting, formatting, or development. Teams should regularly review dependencies for security issues and unnecessary packages.

Project structure

A backend project can separate routes, controllers, services, repositories, middleware, configuration, utilities, and models. The exact structure varies by project size, but the goal is to keep business logic understandable and testable rather than putting everything inside route handlers.

Page 4: Express.js

Express overview

Express is a lightweight Node.js web framework. It provides routing and middleware features for building HTTP APIs. An Express application can define endpoints such as GET, POST, PUT, PATCH, and DELETE and can organize routes into separate routers.

Routes

A route connects an HTTP method and URL pattern to application logic. Path parameters can identify resources, while query parameters can control filtering, pagination, and sorting. Request bodies are commonly used for JSON data in POST, PUT, and PATCH operations.

Middleware

Middleware functions run during the request-response lifecycle. They can authenticate users, validate input, log requests, parse request bodies, add context, or handle errors. Middleware should have a clear responsibility and should call next when the request should continue.

Error handling

A production API should have centralized error handling. Controllers and services can throw meaningful errors, while an error middleware converts them into consistent HTTP responses. Sensitive internal stack traces should not be exposed to clients in production.

Page 5: REST API Design

Resources and HTTP methods

REST APIs commonly model business objects as resources. GET retrieves information, POST creates a resource or triggers an operation, PUT replaces a resource, PATCH partially updates a resource, and DELETE removes a resource when appropriate.

Status codes

Common status codes include 200 for successful requests, 201 for creation, 204 for successful operations without a response body, 400 for invalid requests, 401 for unauthenticated requests, 403 for forbidden operations, 404 for missing resources, and 500 for unexpected server errors.

Validation

Input validation protects APIs from invalid or unexpected data. Validation should happen at the application boundary before business logic is executed. A schema validation library can enforce required fields, data types, ranges, and formats.

Pagination and filtering

Large result sets should normally be paginated. APIs may support parameters such as page, limit, cursor, sort, and filter. Database indexes should be designed around common query patterns so that pagination and filtering remain efficient.

Page 6: Asynchronous Programming

Callbacks

Callbacks are functions passed to other functions and invoked later. They were widely used in older Node.js code. Deeply nested callbacks can make error handling and control flow difficult, which led to broader adoption of Promises and `async/await`.

Promises

A Promise represents a future result. It can be pending, fulfilled, or rejected. Promise chaining allows asynchronous operations to be composed. `Promise.all` can run independent operations concurrently and wait for all results, while `Promise.allSettled` can inspect every result even when some operations fail.

Async/await patterns

`Async/await` makes asynchronous code easier to read. Independent tasks can sometimes be started before awaiting them so they execute concurrently. Sequential awaits should be used when later work depends on earlier results.

Retries and timeouts

Network calls can fail temporarily. Production services often use bounded retries with backoff for appropriate transient failures. Timeouts are also important because an operation that waits forever can consume resources and make failures harder to recover from.

Page 7: Streams and Buffers

Buffers

A Buffer represents binary data in Node.js. Buffers are useful when handling files, network packets, images, compressed data, and other data that is not simply text. They allow Node.js applications to work directly with bytes.

Streams

Streams process data incrementally instead of loading an entire dataset into memory. Readable streams provide data, writable streams consume data, and transform streams modify data as it flows through the pipeline.

Backpressure

Backpressure occurs when a producer generates data faster than a consumer can process it. Node.js streams provide mechanisms that help control this flow. Handling backpressure correctly can prevent excessive memory usage when processing large files or network responses.

Practical use

Streams are useful for uploading and downloading large files, processing logs, transforming CSV data, and sending HTTP responses. For large datasets, streaming is often more memory-efficient than reading the complete dataset at once.

Page 8: Authentication and Security

Authentication

Authentication determines who a user is. Authorization determines what that authenticated user is allowed to do. A typical API may authenticate using a session, access token, or another credential mechanism and then apply authorization rules to protected resources.

JWT

JSON Web Tokens can carry claims used by an API. A common architecture issues a short-lived access token and may use a refresh mechanism to obtain new access tokens. Tokens should be protected appropriately and should not contain unnecessary sensitive information.

Password security

Passwords should never be stored as plain text. Applications should use a password hashing algorithm designed for password storage and should apply appropriate security parameters. Login endpoints should also consider rate limiting and account protection.

API security

Production APIs should validate input, use HTTPS, apply authentication and authorization, protect secrets through environment or secret-management systems, limit request rates where appropriate, and avoid exposing internal errors or credentials in logs and responses.

Page 9: PostgreSQL and Redis

PostgreSQL

PostgreSQL is a relational database commonly used by Node.js backend applications. It supports transactions, constraints, indexes, joins, JSON data, and many advanced database features. Applications should use parameterized queries or safe ORM/query-builder mechanisms to avoid SQL injection.

Connection pooling

Opening a new database connection for every request can be expensive. Connection pools maintain a controlled number of reusable connections. Pool size should be configured according to database capacity, application instances, and workload rather than simply being made as large as possible.

Redis

Redis is an in-memory data store commonly used for caching, sessions, counters, distributed locks, and queues. A cache should have a clear invalidation strategy and appropriate expiration times. Applications should handle cache misses and cache failures safely.

Caching pattern

A common cache-aside pattern checks Redis first. If data is found, the application returns it. If not, the application reads from PostgreSQL, stores an appropriate representation in Redis with an expiration time, and then returns the result.

Page 10: Production Node.js

Logging

Production applications need structured logs containing useful information such as timestamps, log levels, request identifiers, and error context. Sensitive information such as passwords, access tokens, and private user data should not be written to logs.

Environment configuration

Configuration such as database URLs, API keys, and service endpoints should not be hard-coded into source code. Environment variables or a dedicated secrets-management system can provide configuration at deployment time.

Scalability

A Node.js service can be run as multiple instances behind a load balancer. Stateless application design makes horizontal scaling easier. Shared state such as sessions, caches, and job information can be stored in external systems when needed.

Queues and background jobs

Long-running work does not always belong inside an HTTP request. Message queues and background workers can process emails, reports, image processing, scheduled tasks, and other jobs asynchronously. This improves request latency and allows work to be retried independently.

Observability

Production systems benefit from metrics, logs, and traces. Useful metrics include request latency, error rate, throughput, CPU and memory usage, database performance, and queue depth. Health checks can help load balancers determine whether an instance is ready to receive traffic.